



NATIONAL SENIOR CERTIFICATE EXAMINATION
NOVEMBER 2025

INFORMATION TECHNOLOGY: PAPER I

MARKING GUIDELINES

Time: 3 hours

150 marks

These marking guidelines are prepared for use by examiners and sub-examiners, all of whom are required to attend a standardisation meeting to ensure that the guidelines are consistently interpreted and applied in the marking of candidates' scripts.

The IEB will not enter into any discussions or correspondence about any marking guidelines. It is acknowledged that there may be different views about some matters of emphasis or detail in the guidelines. It is also recognised that, without the benefit of attendance at a standardisation meeting, there may be different interpretations of the application of the marking guidelines.

SECTION A SQL**QUESTION 1****Question 1.1 (5)**

```
SELECT *  
FROM tblClients  
WHERE Loyalty = 3 AND Province <> "Gauteng"  
Accept NOT Province = 'Gauteng'
```

Question 1.2 (6)

```
SELECT ProdName, Fee  
FROM tblProducts  
WHERE (ProdName LIKE '*Premium' OR PRODNAME LIKE '*Deluxe') AND FEE  
> 400  
Accept ProdName LIKE '%Premium' OR PRODNAME LIKE '%Deluxe'
```

Question 1.3 (3)

```
SELECT COUNT(*) AS NumSales  
FROM tblSales
```

Question 1.4 (5)

```
SELECT License, SaleDate  
FROM tblSales  
WHERE YEAR (NOW()) - YEAR(SaleDate) <= 1  
ORDER BY License  
Accept NOW, CURRENT_DATE or DATE() in place of NOW()
```

Accept LEFT(SaleDate, 4) or SUBSTR(SaleDate, 1, 4) in place of
YEAR(SaleDate)

Accept variations, such as WHERE YEAR(NOW()) <= 1 + YEAR(SaleDate)

Question 1.5 (4)

```
SELECT ClientName, License  
FROM tblSales, tblClients  
WHERE tblSales.ClientID = tblClients.ClientID
```

Alternate Solution:

```
SELECT ClientName, License  
FROM tblSales INNER JOIN tblClients  
ON tblSales.ClientID = tblClients.ClientID
```

Question 1.6 (5)

```
SELECT ProdName, Fee  
FROM tblProducts  
WHERE Fee > (SELECT AVG(Fee) FROM tblProducts)
```

Question 1.7 (7)

```
SELECT Province, SUM(Fee) AS ValueOfSales
FROM tblProducts, tblClients, tblSales
WHERE tblProducts.ProdID = tblSales.ProdID AND tblSales.ClientID =
tblClients.ClientID
GROUP BY Province
HAVING SUM(FEE) > 1000
```

Alternate solution:

```
SELECT Province, SUM(Fee) AS ValueOfSales
FROM tblProducts INNER JOIN (tblClients INNER JOIN tblSales
ON tblSales.ClientID = tblClients.ClientID) ON tblProducts.ProdID
= tblSales.ProdID
GROUP BY Province
HAVING SUM(FEE) > 1000
```

Question 1.8 (8)**Access:**

```
UPDATE tblProducts
SET ProdName = LEFT(ProdName, LEN(ProdName)- 7) & ' Premium'
WHERE ProdName LIKE ("*Deluxe")
```

MySQL:

```
UPDATE tblProducts
SET ProdName = CONCAT (LEFT (ProdName, LENGTH(ProdName) - 7) , '
Premium')
WHERE ProdName LIKE ("%Deluxe")
```

JavaDB:

```
UPDATE tblProducts
SET ProdName = SUBSTR (ProdName, 1, LENGTH(ProdName) - 7) || '
Premium')
WHERE ProdName LIKE ("%Deluxe")
```

Accept variations such as `LEN(ProdName)-6) & 'Premium'`

Accept `WHERE RIGHT(Prodname, 6) = 'Deluxe'`

Question 1.9 (7)

```
INSERT INTO tblSales
VALUES (14, 25, INT (RND()*1000001+1000000), #2025/03/25#)
Accept '2025/03/25'
```

SECTION B**JAVA SOLUTION****QUESTION 2**

```
//Question 2.1 (3)
//class header
public class Free {
    //all fields private
    //all fields correctly typed with correct names
    private String name;
    private String email;
    private LocalDate actDate;

    //Question 2.2 (4)
    //correct header with parameter names, types and order
    public Free(String inN, String inE, String inA) {
        //set name and email fields to parameters
        name = inN;
        email = inE;
        //create formatter with correct format
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd MM
            yyyy");
        //parse date (award even without parameter)
        actDate = LocalDate.parse(inA, dtf);
    }

    //Question 2.3 (1)
    //completely correct accessor
    public String getName() {
        return name;
    }

    //Question 2.4 (1)
    //completely correct mutator
    public void setEmail(String inE) {
        email = inE;
    }

    //Question 2.5 (3)
    public boolean isActive() {
        //Add six months to activation date
        LocalDate temp = actDate.plusMonths(6);
        //return if current date equals adjusted activation date
        //OR if current date is before adjusted activation date
        return (LocalDate.now().equals(temp) ||
            LocalDate.now().isBefore(temp));
    }
    //Accept correct variations, such as
    actDate.isAfter(LocalDate.now().minusMonths(6))
}
```

```
//Alternate version
public boolean isActive() {
    //Calculate period of time between
    //Using activation date and current date in correct order
    Period actTime = Period.between (actDate, now);
    //Check that the period is less than or equal to 6 months
    return actTime.getMonths() <= 6;
}

//Question 2.6 (2)
public boolean authorise() {
    //Set actDate to the current date
    actDate = LocalDate.now();
    //return true
    return true;
}

@Override
//Question 2.7 (2)
//Correct method header
public String toString() {
    //return all fields separated by spaces
    return name + " " + email + " " + actDate;
}
}
```

QUESTION 3

```
//Question 3.1 (3)
//class header extending Free
public class Premium extends Free {

    //all fields private
    //all fields correctly typed and named
    private String licNo;
    private int qty;
    private String[] macArr;

    //Question 3.2 (6)
    //correct method header with correct parameter types, names and
        order
    public Premium(String inN, String inE, String inA, String inL,
        int inQ) {
        //call to super with correct parameters as first line in
            class
        super(inN, inE, inA);
        //assign field names
        licNo = inL;
        qty = inQ;
        //instantiate array to qty size
        macArr = new String[qty];
        //loop through entire array
        for (int i = 0; i < qty; i++) {
            //set element to "Available", award if using macArr[qty]
            macArr[i] = "Available";
        }
    }

    @Override
    //Question 3.3 (3)
    //correct method header
    public String toString() {
        //call to super.toString()
        //append remaining licNo and Qty with spaces
        return super.toString() + " " + licNo + " " + qty;
    }
}
```

QUESTION 4

```
//Question 4.1 (1)
//Correct class header
public class RequestManager {

    //Question 4.2 (3)
    //declare private array
    //set array to size 40
    private Free[] accArr = new Free[40];
    //declare private size, not necessary to initialise to 0
    private int accSize = 0;
    Only penalise ONCE for lack of private

    //Question 4.3 (2)
    //both constants declared as static, final and all caps
    //set correct values
    public static final int ACCESS = 0;
    public static final int AUTHORISE = 1;

    //Question 4.4 (11)
    public RequestManager() {
        try {
            //Create scanner or BufferedReader for file
            Scanner scFile = new Scanner(new
            File("Accounts.txt"));
            //Loop through entire file (may use scFile.next())
            while (scFile.hasNextLine()) {
                //Read line from file
                String line = scFile.nextLine();
                //Scanner with correct delimiter OR .split("#")
                Scanner scLine = new
                Scanner(line).useDelimiter("#");
                //Extract first three tokens
                String name = scLine.next();
                String email = scLine.next();
                String actDate = scLine.next();
                //Check for extra field
                if (scLine.hasNext()) {
                    String licNo = scLine.next();
                    int qty = scLine.nextInt();
                    //create Premium object
                    //at correct position (if not awarded here,
                    may award for Free object)
                    accArr[accSize] = new Premium(name, email,
                    actDate, licNo, qty);
                    //otherwise
                } else {
                    //create Free object
```

```
        accArr[accSize] = new Free(name, email,
actDate);
    }
    //Increment accSize
    accSize++;
    scLine.close();
}
scFile.close();
} catch (FileNotFoundException ex) {
    System.out.println("Error. File not found.");
}
}

//Question 4.5 (5)
public void sortByName() {
    //Outer loop with correct limits follow through if accSize
    //decremented in constructor
    for (int i = 0; i < accSize - 1; i++) {
        //Inner with correct limits
        for (int j = i + 1; j < accSize; j++) {
            //Compare names using compareTo and >0
            if
(accArr[i].getName().compareTo(accArr[j].getName()) > 0) {
                //Creation of temp variable
                Free temp = accArr[i];
                //correct swap into i and into j
                accArr[i] = accArr[j];
                accArr[j] = temp;
            }
        }
    }
}

@Override
//Question 4.6 (4)
public String toString() {
    //Create string to return
    String ret = "";
    //Loop through entire array
    for (int i = 0; i < accSize; i++) {
        //Append current element to array
        ret = ret + accArr[i] + "\n";
    }
    //Return some attempt at correct string
    return ret;
}
```

QUESTION 6.1, 6.2 & 6.3

//Question 6.1 (4)

```
public boolean isAuthorised(String mac) {  
    //Loop through array  
    for (int i = 0; i < qty; i++) {  
        //Check if current element = parameter  
        if (macArr[i].equals(mac)) {  
            //Return true correctly  
            return true;  
        }  
    }  
    //Return false correctly  
    return false;  
}
```

//Question 6.2 (4)

```
public boolean authorise(String mac) {  
    int i = 0;  
    boolean found = false;  
    //While loop through array  
    while (i < qty && !found) {  
        //Check if current element = "Available"  
        if (macArr[i].equals("Available")) {  
            //Set mac value AND authorise (or set actDate  
            //through another method)  
            macArr[i] = mac;  
            super.authorise();  
            return true;  
            found = true;  
        }  
        i++;  
    }  
    //Return true correctly AND return false correctly  
    return found;  
}
```

```
//Question 6.3 (4)
public boolean isValid() {
    String name = this.getName();
    //Create sum variable
    int sum = 0;
    //Loop through name string AND add char value
    for (int i = 0; i < name.length(); i++) {
        sum = sum + name.charAt(i);
    }
    //Find modulo 100
    int check = sum % 100;
    //Return true/false correctly
    return check == Integer.parseInt(licNo.substring(4));
}
```

QUESTION 7.1, 7.2 & 7.3

//Question 7.1 (9)

```
//Correct header with int and Free parameters and private
private String processFree(int action, Free account) {
    //Switch or nested if on action
    switch (action) {
        //equals to ACCESS (must use constant)
        case ACCESS:
            //Check if active
            if (account.isActive()) {
                //Correct message
                return account.getName() + " has access to
the free version.";
            } else {
                //Correct message
                return account.getName() + " has an
expired account.";
            }
            //equals to AUTHORISE (must use constant)
        case AUTHORISE:
            //Call authorise
            account.authorise();
            //Set correct message
            return account.getName() + " has been
authorised for the free version.";
        }
        return ""//or empty
    }
}
```

//Question 7.2 (10)

```
//Correct header, with all three parameters of the correct
types and private.
```

```
private String processPremium(int action, Premium account,
String mac) {
    String ret = "";
    //Check if account/licenseNo. is valid
    if (account.isValid()) {
        //switch or nested inside if
        switch (action) {
            case ACCESS:
                //Check account is active and authorised
                if (account.isActive() &&
account.isAuthorised(mac)) {
                    //Correct message
                    ret = " " + mac + " has access to the
premium version.";
                } else {
                    //Correct message
                    ret = " " + mac + " does not have access
to the premium version.";
                }
                break;
            }
        }
    }
}
```

```

        case AUTHORISE:
            //Call authorise
            if (account.authorise(mac)) {
                //Correct message if authorise succeeded
                ret = " " + mac + " has been authorised
for the premium version.";
            } else {
                //Correct message if authorise failed
                ret = " " + mac + " is not authorised.
Purchase another license.";
            }
            break;
        }
    } else {
        //Correct message if license invalid
        ret = " has an invalid license.";
    }
    return ret;
}

```

NOTE: Penalise only ONCE for use of 0/1 instead of ACCESS/AUTHORISE

```

//Question 7.3 (10)
public String processRequests() {
    String ret = "";
    try {
        //Scanner for file
        Scanner scFile = new Scanner(new
File("requests.txt"));
        //Loop through file
        while (scFile.hasNextLine()) {
            String line = scFile.nextLine();
            //Extract elements of line
            Scanner scLine = new
Scanner(line).useDelimiter("#");
            int action = scLine.nextInt();
            String name = scLine.next();
            //Loop through array
            int i = 0;
            boolean found = false;
            while (i < accSize && !found) {
                //Check if current element equals name from
file

```

```
        if (accArr[i].getName().equals(name)) {
            found = true;
            //Check if object is Premium
            if (accArr[i] instanceof Premium) {
                //Read mac INSIDE if
                String mac = scLine.next();
                //Call processPremium correctly
                ret = ret + name + ' ' +
processPremium(action, (Premium)accArr[i], mac) + "\n";
            } else {
                //Call processFree correctly
                ret = ret + name + processFree(action,
accArr[i]) + "\n";
            }
        }
        scLine.close();
    }
    scFile.close();
} catch (FileNotFoundException ex) {
    System.out.println("Error. File not found.");
}
//Return some attempt at final output
return ret;
}
```

QUESTION 5 & 7.4

```
//Question 5.1 (1)
//Create class with main method
public class RequestUI {

    public static void main(String[] args) {
        //Question 5.2 (1)
        //Instantiate object, here or as field
        RequestManager rm = new RequestManager();
        //Question 5.3 (1)
        //Call sortByName
        rm.sortByName();
        //Question 5.4 (1)
        //Print array
        System.out.println(rm);

        //Question 7.4 (1)
        //Print result of processRequests
        System.out.println(rm.processRequests());
    }
}
```

Total: 150 marks

DELPHI SOLUTION**QUESTION 2**

```

//Question 2.1 (3)
//class header
unit Free;
    //all fields private
    //all fields correctly typed with correct names

interface
uses SysUtils, DateUtils;
type TFree = class
    private
        name : string;
        email : string;
        actDate : TDate;

    public
        constructor Create(inN : string; inE : string; inA : string);
        function getName() : string;
        procedure setEmail(inE : string);
        function isActive() : Boolean;
        function authorise() : Boolean;
        function toString() : string; virtual;
end;

implementation

    //Question 2.2 (4)
    //correct header with parameter names, types and order
    constructor TFree.Create(inN : string; inE : string; inA :
        string);
    var
        y, m, d: string;
    begin
        //set name and email fields to parameters
        name := inN;
        email := inE;
        //correctly extract day, month and year from inA
        d := Copy(inA, 1, Pos(' ', inA) -1);
        Delete(inA, 1, Pos(' ', inA));
        m:= Copy(inA, 1, Pos(' ', inA) -1);
        Delete(inA, 1, Pos(' ', inA));
        y := inA;
        //encode date
        actDate := EncodeDate(StrToInt(y), StrToInt(m),
            StrToInt(d));
    end;

```

```
//Question 2.3 (1)
//completely correct accessor
function TFree.getName: string;
begin
    Result := name;
end;

//Question 2.4 (1)
//completely correct mutator
procedure TFree.setEmail(inE: string);
begin
    email := inE;
end;

//Question 2.5 (3)
function TFree.isActive: Boolean;
var monthDiff : Integer;
begin
    //Calculate months between current date and actDate
    monthDiff := DateUtils.MonthsBetween(Date, actDate);
    //return false if months is greater than 6
    //otherwise return true
    if monthDiff > 6 then
        begin
            Result := false;
        end
    else
        begin
            Result := true;
        end;
    end;
end;

//Question 2.6 (2)
function TFree.authorise: Boolean;
begin
    //Set actDate to the current date
    actDate := Date;
    //return true
    Result := true;
}

//Question 2.7 (2)
//Correct method header
function TFree.toString: string;
begin
    //return all fields separated by spaces
    Result := name + ' ' + email + ' ' + DateToStr(actDate);
end;

end.
```

QUESTION 3

```

unit Premium;

interface
uses SysUtils, StrUtils, Free;

//Question 3.1 (3)
//TPremium inherits from TFree
type TPremium = class(TFree)

    //all fields private
    //all fields correctly typed and named
private
    licNo : string;
    qty : Integer;
    macArr : array of string;

public
    constructor Create(inN : string; inE : string; inA : string;
                      inL : string; inQ : Integer);
    function authorise(mac : string) : boolean;
    function isAuthorised(mac : string) : boolean;
    function isValid() : boolean;
    function toString() : string; override;
end;

implementation

//Question 3.2 (6)
//correct method header with correct parameter types, names and
//order
constructor TPremium.Create(inN: string; inE: string; inA:
                           string; inL: string; inQ: Integer);
var
    i: Integer;
begin
    //call to super with correct parameters as first line in
    //class
    Inherited Create(inN, inE, inA);
    //assign field names
    licNo := inL;
    qty := inQ;
    //instantiate array to qty size
    SetLength(macArr, qty);
    //loop through entire array
    for i := 0 to qty - 1 do
        begin
            //set element to "Available"
            macArr[i] := 'Available';
        end;
    end;
end;

@Override

```

```
//Question 3.3 (3)
//correct method header
function TPremium.toString : string;
    //call to super.toString()
    //append remaining licNo and Qty with spaces
    Result := Inherited toString + ' ' + licNo + ' ' +
        IntToStr(qty);
end;
```

QUESTION 4

```
unit RequestManager;

interface
uses SysUtils, Free, Premium;
//Question 4.1 (1)
//Correct class header
type TRequestManager = class {

    //Question 4.2 (3)
    //declare private array
    //set array to size 40
private
    accArr : array[1..40] of TFree;
    //declare private size
    accSize : Integer;

    Only penalise ONCE for lack of private

    //Question 4.3 (2)
    //both constants declared as static, final and all caps
    //set correct values
public
    const
        ACCESS = 0;
        AUTHORISE = 1;

    constructor Create();
    procedure sortByName();
    function toString() : string; override;
    function processRequests() : string;

end;

implementation

    //Question 4.4 (11)
    constructor TRequestManager.Create();
    var
        inFile : textfile;
        line, name, email, actDate, licNo, qty : string;
    begin
    if FileExists('Accounts.txt') <> true then
        begin
            WriteLn('Error. File not found');
        end
    else
        begin
```

```

//Open file
AssignFile(inFile, 'Accounts.txt');
Reset(inFile);
accSize := 0;
//Loop through entire file
while NOT EOF(inFile) do
begin
    //Read line from file
    ReadLn(inFile, line);
    //Increment accSize
    Inc(accsize);
    //Extract first two tokens
    name := Copy(line, 1, Pos('#', line) - 1);
    Delete(line, 1, Pos('#', line));
    email := Copy(line, 1, Pos('#', line) - 1);
    Delete(line, 1, Pos('#', line));
    //Check for extra field
    if Pos('#', line) > 0 then
    begin
        actDate := Copy(line, 1, Pos('#', line) - 1);
        Delete(line, 1, Pos('#', line));
        licNo := Copy(line, 1, Pos('#', line) - 1);
        Delete(line, 1, Pos('#', line));
        qty := line;
        //create Premium object
        //at correct position (if not awarded here,
may award for Free object
        accArr[accSize] := TPremium.Create(name,
email, actDate, licNo, StrToInt(qty));
        end
        //otherwise
        else
        begin
            //create Free object
            actDate := line;
            accArr[accSize] := TFree.Create(name, email,
actDate);
        end;
    end;
end;
end;

//Question 4.5 (5)
procedure TRequestManager.sortByName();
var
    i, j : Integer;
    temp : TFree;
begin

```

```
//Outer loop with correct limits
for i := 1 to accSize do
begin
    //Inner with correct limits
    for j := i to accSize - 1 do
begin
    //Compare names correctly
    if CompareStr(accArr[j].getName(), accArr[j +
1].getName()) > 0 then
begin
    //Creation of temp variable
    temp := accArr[j];
    //correct swap of variables
    accArr[j] := accArr[j + 1];
    accArr[j + 1] := temp;
end;
end;
end;
end;

@Override
//Question 4.6 (4)
function TRequestManager.toString() : string;
var
    i : Integer;
    //Create string to return
    ret : string;
begin
    ret := '';
    //Loop through entire array
    for i := 1 to accSize do
begin
    //Append current element to array
    ret := ret + accArr[i].toString + #13#10;
end;
    //Return some attempt at correct string
    Result := ret;
end;
```

QUESTION 6.1, 6.2 & 6.3

```
//Question 6.1 (4)
function TPremium.isAuthorised(mac: string): Boolean;
var
    found : Boolean;
    i : integer;
begin
    found := false;
    //Loop through array
    for i := 0 to qty - 1 do
        begin
            //Check if current element = parameter
            if macArr[i] = mac then
                begin
                    //Set found to true
                    found := true;
                }
            }
        //Return true/false correctly
        Result := found
    }

//Question 6.2 (4)
function TPremium.authorise(mac: string): Boolean;
var
    found : Boolean;
    i : Integer;
begin
    i := 0;
    found := false;
    //While loop through array
    while (i < qty) and (found = false) do
        begin
            //Check if current element = "Available"
            if macArr[i] = 'Available' then
                begin
                    //Set mac value AND authorise (or set actDate
                    //through another method)
                    macArr[i] := mac;
                    Inherited authorise();
                    found := true;
                end
            end;
            Inc(i);
        end;
        //Return true/false correctly
        Result := found;
    end;
```

//Question 6.3 (4)

```
function TPremium.isValid() : Boolean;
var
  i, sum : Integer;
  name : string;
begin
  name := Inherited getName();
  //Initialise sum variable
  sum := 0;
  //Loop through name string AND add char value
  for i := 1 to Length(Inherited getName()) do
  begin
    sum := sum + Ord(Inherited getName[i]);
  end;
  //Find modulo 100
  sum := sum mod 100;
  //Return true/false correctly
  if RightStr(licNo, 2) = IntToStr(sum) then
  begin
    Result := true;
  end
  else
  begin
    Result := false;
  end;
end;
end.
```

QUESTION 7.1, 7.2 & 7.3

//Question 7.1 (9)

//Correct header with int and Free parameters (must be declared as private in interface).

function TRequestManager.processFree(action: Integer; account: TFree): string;

var

output : string;

//nested if statement OR switch on action

//use of constant for ACCESS

if action = ACCESS then {

//Check if active

if account.isActive() then

begin

//correct message

output := ' has access to the free version.'

end

else

begin

//correct message

output := ' has an expired account.'

end;

end

else

//equals to AUTHORISE (must use constant)

if action = AUTHORISE

begin

//Call authorise

account.authorise();

//Set correct message

output := ' has been authorised for the free version.';

end;

Result := output;

End;

//Question 7.2 (10)

//Correct header, with all three parameters of the correct types must be declared as private in interface.

function TRequestManager.processPremium(action: Integer;

account: TPremium; mac: string) : string;

var

output : string;

begin

output := "";

//Check if account/licenseNo. is valid

if account.isValid = true then

begin

//switch or nested if

if action = ACCESS then

begin

```
//Check account is active and authorised
if (account.isActive()) AND (account.isAuthorised(mac))
then
  begin
    output := mac + ' has access to the premium version.';
  end
else
  begin
    //Correct message inside if AND else
    output := mac + ' does not have access to the premium
version.';
  end;
else if action = AUTHORISE then
  begin
    //Call authorise
    if account.authorise(mac) then
      begin
        //Correct message if authorise succeeded
        output := mac + ' has been authorised for the premium
version.';
      end
    else
      begin
        //Correct message if authorise failed
        output := mac + ' is not authorised. Purchase another
license.';
      end;
    end;
  end
else
  begin
    //Correct message if license invalid
    output := 'has an invalid license.';
  end;
  Result := output;
end;
```

NOTE: Penalise only ONCE for use of 0/1 instead of ACCESS/AUTHORISE

```

//Question 7.3 (10)
function TRequestManager.processRequests: string;
var
    i : Integer;
    found : Boolean;
    inFile : textfile;
    temp : TPremium;
    output, line, action, name, mac : string;
begin
    output := '';
    if FileExists('Requests.txt') <> true then
begin
    WriteLn('Error. File not found.');
```

end

```

else
begin
    //Openfile
    AssignFile(inFile, 'Requests.txt');
```

Reset(inFile);

```

    //Loop through file
    while NOT EOF(inFile) do
begin
    ReadLn(inFile, line);
    //Extract elements of line
    action := Copy(line, 1, Pos('#', line) -1);
    Delete(line, 1, Pos('#', line));
    if Pos('#', line) > 0 then
begin
        name := Copy(line, 1, Pos('#', line) -1);
        Delete(line, 1, Pos('#', line));
    end
    else
begin
        name := line;
    end;
    //Loop through array
    i := 0;
    found := false;
    while (i <= accSize) AND (found = false) do
begin
    Inc(i);
    //Check if current element equals name from
file
        if name = accArr[i].getName() then
begin
            found := true;
            //Check if object is Premium
            if accArr[i] is TPremium then
begin
```

```
        //Read mac INSIDE if
        mac := line;
        //Call processPremium correctly
        temp := accArr[i] as TPremium;
        output := output + name + ' ' +
processPremium(StrToInt(action), temp, mac) + #13#10;
    end
    else
    begin
        //Call processFree correctly
        output := output + name +
processFree(StrToInt(action), accArr[i]) + #13#10;
    end;
end;
end;
//Return some attempt at final output
Result := output;
```

QUESTION 5 & 7.4

```
//Question 5.1 (1)
//Create program
program RequestUI; {
{$APPTYPE CONSOLE}

{$R *.res}

uses
    System.SysUtils,
    Free in 'Free.pas',
    Premium in 'Premium.pas',
    RequestManager in 'RequestManager.pas';

var rm : TRequestManager;
begin
    try
        //Question 5.2 (1)
        //Instantiate object
        rm := TRequestManager.Create();
        //Question 5.3 (1)
        //Call sortByName
        rm.sortByName();
        //Question 5.4 (1)
        //Print array
        WriteLn(rm.toString());

        //Question 7.4 (1)
        //Print result of processRequests
        WriteLn(rm.processRequests());
        ReadLn;
    except
        on E: Exception do
            Writeln(E.ClassName, ': ', E.Message);
        end;
    end.
end.
```

Total: 150 marks